VIGILADA MINEDUCACIÓN

UNIVERSIDAD DEL QUINDÍO
Res.MEN 014915 - 02 AGO 2022
RENOVACIÓN ACREDITACIÓN

LittleStyle

Software Architecture Description – C4 Model

Autor:
Valentina González Diaz
Valentina Rodriguez Castro
Oscar Tomas Aristizabal Velasquez



Versión: 3.0
Fecha: 01/11/26

UNIQUINDÍO
en conexión territorial
www.uniquindio.edu.co

1. RESUMEN ARQUITECTÓNICO

LittleStyle es una plataforma digital de e-commerce para ropa infantil, cuyo propósito es centralizar y automatizar la gestión de perfiles infantiles, la recomendación personalizada de tallas, la compra de prendas y el control de inventario mediante canales web diferenciados por rol. El cliente (padre, madre o acudiente) gestiona perfiles infantiles, consulta el catálogo y realiza pedidos; el vendedor administra su catálogo e inventario; y el administrador supervisa la plataforma y gestiona usuarios. El sistema se integra con una pasarela de pago externa para procesar las transacciones.

1.1 Usuarios principales

- **Cliente(Padre,madre o acudiente):**gestiona perfiles infantiles, consulta el catálogo con recomendación de talla, administra el carrito y hace seguimiento a sus pedidos.
- **Vendedor:** administra su catálogo de prendas, controla el stock, recibe alertas de reabastecimiento y consulta el dashboard de su propio negocio, apela decisiones de moderación y exporta reportes de su tienda.
- **Administrador:** gestiona las cuentas de usuarios (habilitar, suspender o bloquear), modera las publicaciones del catálogo, atiende incidencias y apelaciones, consulta la auditoría de sus acciones y los dashboards globales de la operación.

1.2 Problema que resuelve

LittleStyle resuelve la dificultad de los padres para elegir tallas y prendas adecuadas para sus hijos, dada la variabilidad del crecimiento infantil y posibles alergias textiles. Además, centraliza para los vendedores la administración de productos e inventario, e integra un pago seguro mediante un tercero, mejorando la experiencia del cliente y la eficiencia operativa.

1.3 Alcance

¿Qué incluye?

- Registro y autenticación de clientes, vendedores y administradores.
- Gestión de perfiles infantiles (medidas, alergias, preferencias).
- Recomendación personalizada de tallas y prendas.
- Consulta del catálogo con filtros avanzados.
- Gestión del carrito de compras y pedidos.
- Seguimiento del estado del pedido.
- Integración con pasarela de pago externa.
- Administración de productos e inventario, con alertas de stock bajo.
- Gestión de cuentas, moderación posterior de publicaciones, incidencias y auditoría administrativa. Dashboards de ventas, inventario y demanda; reportes descargables en PDF,XLSX y CSV.
- Notificaciones por correo (pedidos, stock bajo, estado de cuenta y moderación).

¿Qué **NO** incluye?

- Gestión interna de manufactura o confección textil.
- Operación logística/transporte directo de paquetería externa (solo se actualizan estados y número de guía).

1.4 Procesos involucrados en el sistema LittleStyle

- **Proceso de gestión del perfil infantil y recomendación de tallas**

  Objetivo: permitir al cliente gestionar su cuenta, registrar y administrar perfiles infantiles (hasta 10 por cliente) para apoyar la recomendación de tallas y prendas.

  Incluye:
  - Registro y actualización de medidas, con validación e historial de crecimiento.
  - Selección de alergias textiles y preferencias de estilo.
  - Cifrado y almacenamiento seguro de los datos del menor.
  - Cálculo del porcentaje de completitud del perfil.
  - Edición y eliminación de la cuenta (con sus datos asociados).

- **Proceso de gestión del catálogo,compras,pedidos e inventario**

  Objetivo: permitir al vendedor administrar su catálogo, su inventario y el despacho de pedidos.

  Incluye:
  - Registro y actualización de prendas (fotos, precios, tallas y composición textil).
  - Stock y nivel mínimo por talla, con alertas automáticas de stock bajo por correo.
  - Gestión de despachos por vendedor: estados “en preparación” y “enviado”, con número de guía.

  Objetivo: permitir al cliente explorar, comprar y hacer seguimiento a su pedido.

  Incluye:
  - Búsqueda, filtrado y recomendación de talla.
  - Carrito de compras y resumen del pedido (subtotal más costo de envío).
  - Pago mediante la pasarela externa Wompi.
  - Creación del pedido solo tras el pago aprobado, con descuento atómico del stock.
  - Seguimiento del estado del pedido y notificación por correo con la guía.

- **Proceso de gestión administrativa y procesos internos**

  Objetivo: garantizar la administración y moderación de cuentas y publicaciones, la atención de incidencias y apelaciones y la consolidación de información para la toma de decisiones.

  Incluye:
  - Gestión de cuentas: habilitar, suspender, bloquear y eliminar con motivo obgilatorio.
  - Moderación posterior de publicaciones: aprobar, desactivar con motivo o restaurar prendas del catálogo.
  - Atención de incidencias y apelaciones con respuesta al usuario y notificación por correo electrónico.
  - Auditoría verificable de las acciones administrativas mediante cadena de hashes SHA-256.
  - Cancelación de pedidos que aún no han sido enviados..
  - Dashboards de ventas, demanda por talla, productos más vendidos y más consultados, inventario disponible y stock bajo, con alcance según el rol.

- Exportación de reportes de ventas, demanda, inventario y búsquedas en PDF, XLSX o CSV, con límite de 5000 filas por reporte.

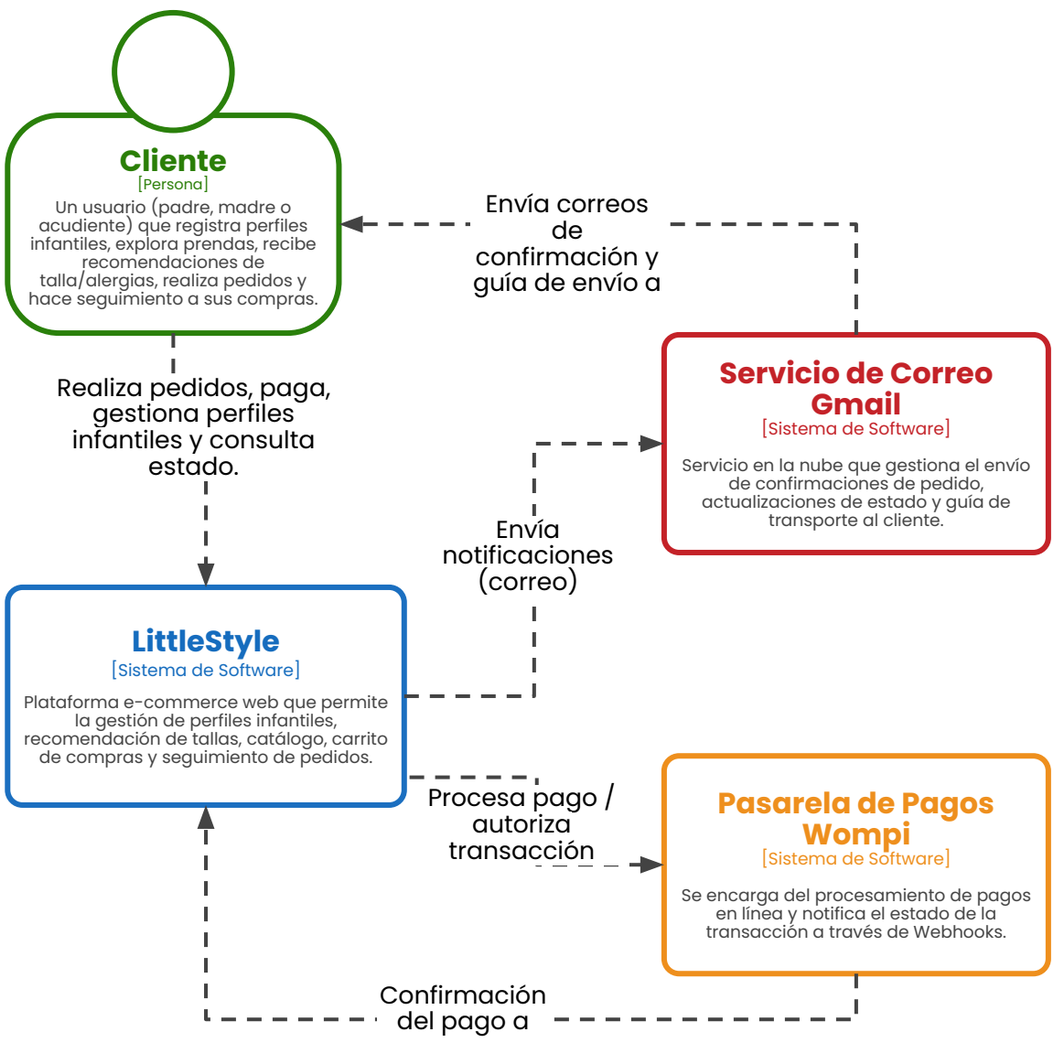
2. C4MODEL

El sistema **LittleStyle** contempla las interacciones de tres roles de usuario principales (Cliente, Vendedor y Administrador) y servicios externos integrados. El modelo C4 representa el alcance completo de la plataforma, detallando el frontend, backend, la base de datos y las integraciones externas. Cabe aclarar que las operaciones logísticas externas de paquetería y manufactura no forman parte del alcance directo del sistema, limitándose la plataforma al registro de guías de envío y actualización de estados del pedido.

2.1 Diagrama de contexto (N1)

Presenta el sistema LittleStyle como "caja negra", usuarios y sistemas externos.

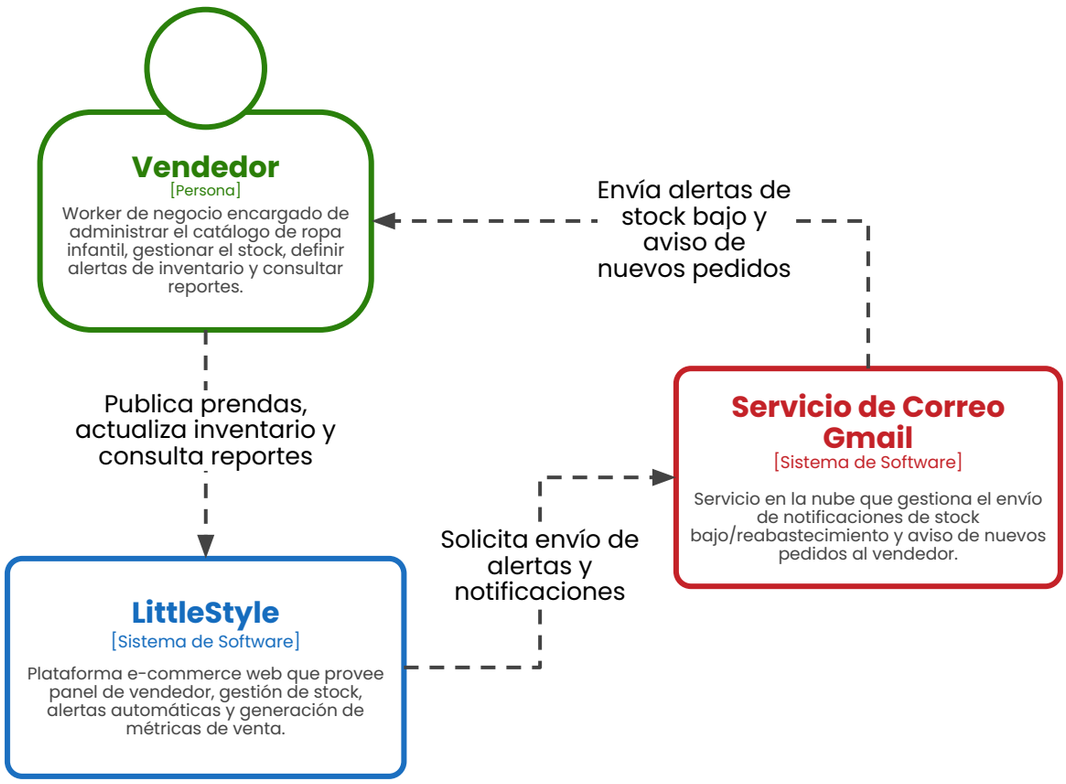
- **Cliente (Padre, madre o acudiente)**



2.2 Diagrama de contexto (N2)

Presenta el sistema LitlleStyle como “caja negra”, usuarios y sistemas externos.

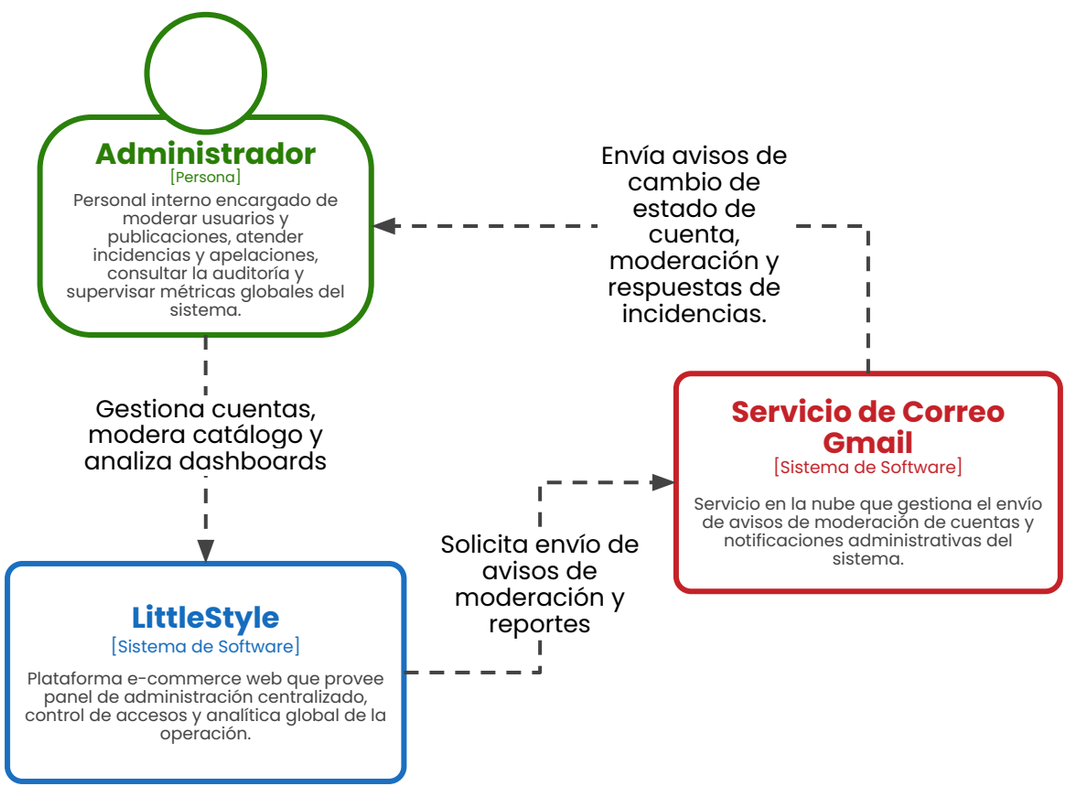
- **Vendedor**



2.3 Diagrama de contexto (N3)

Presenta el sistema LitlleStyle como “caja negra”, usuarios y sistemas externos.

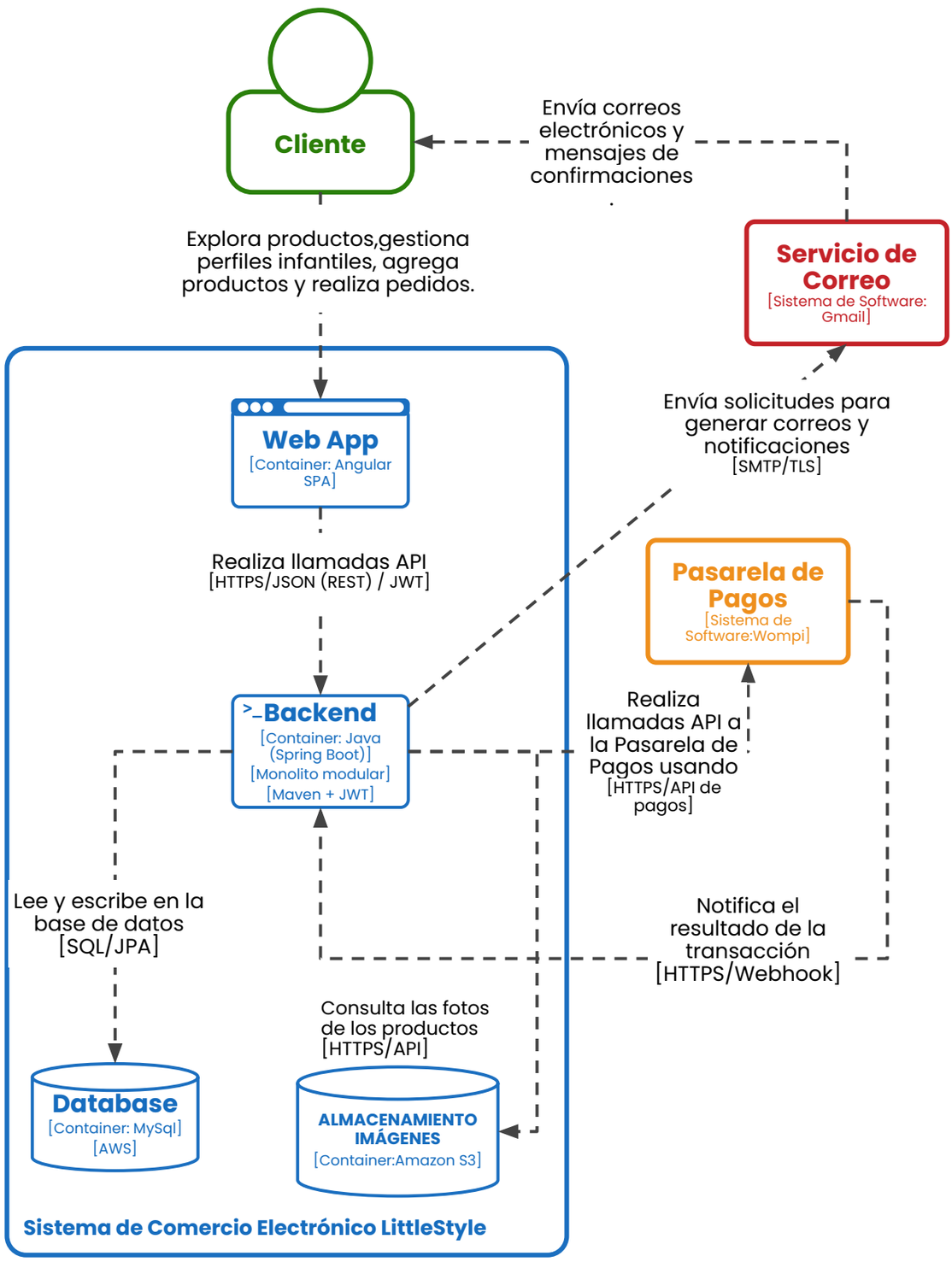
- **Administrador**



2.4 Diagrama de contenedores (N1)

Presenta "qué partes ejecutables" tiene el sistema LittleStyle (web, API, BD, app móvil, colas, etc.).

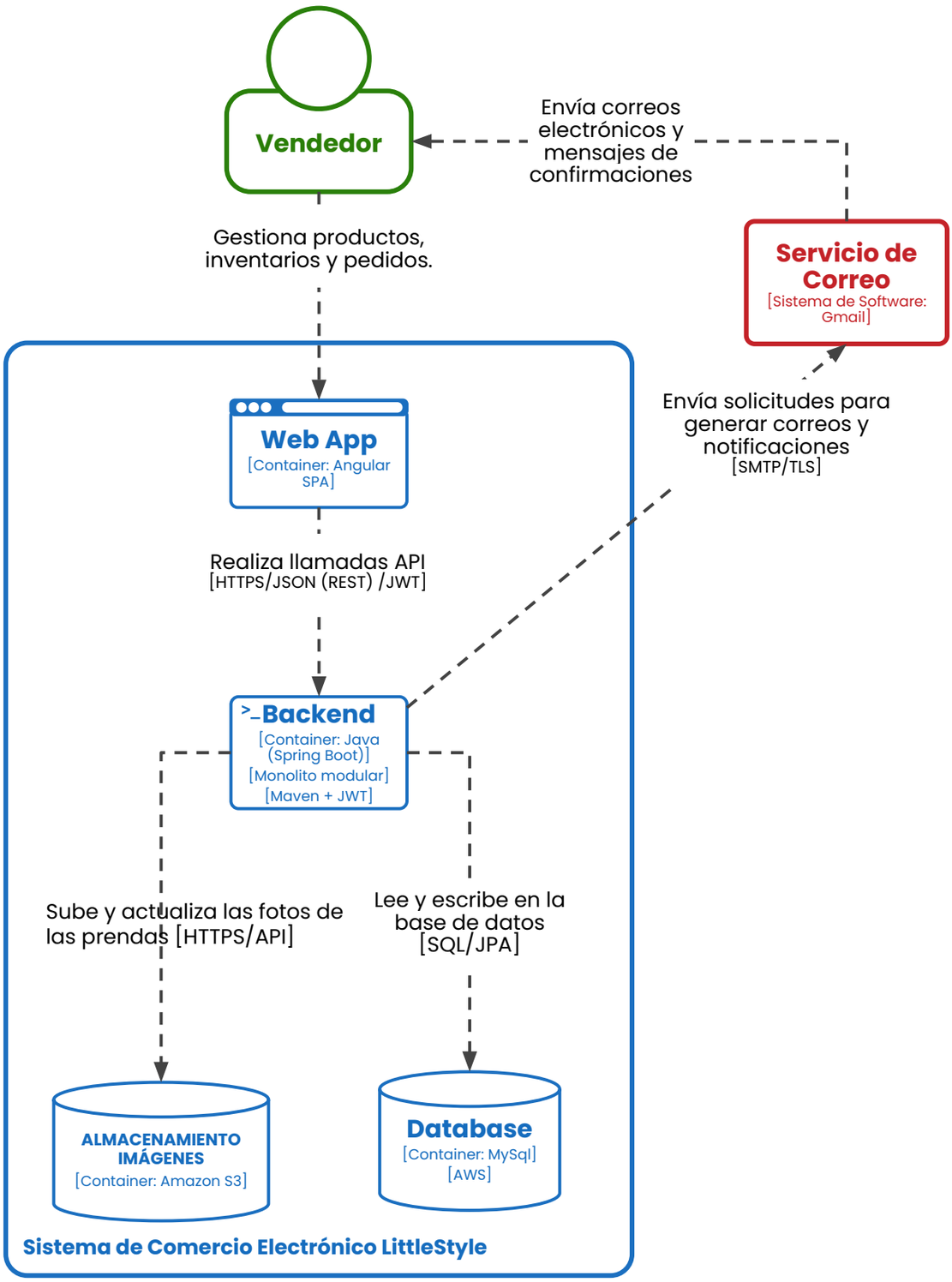
- **Cliente(Padre,madre o acudiente)**



2.5 Diagrama de contenedores (N2)

Presenta "qué partes ejecutables" tiene el sistema LittleStyle (web, API, BD, app móvil, colas, etc.).

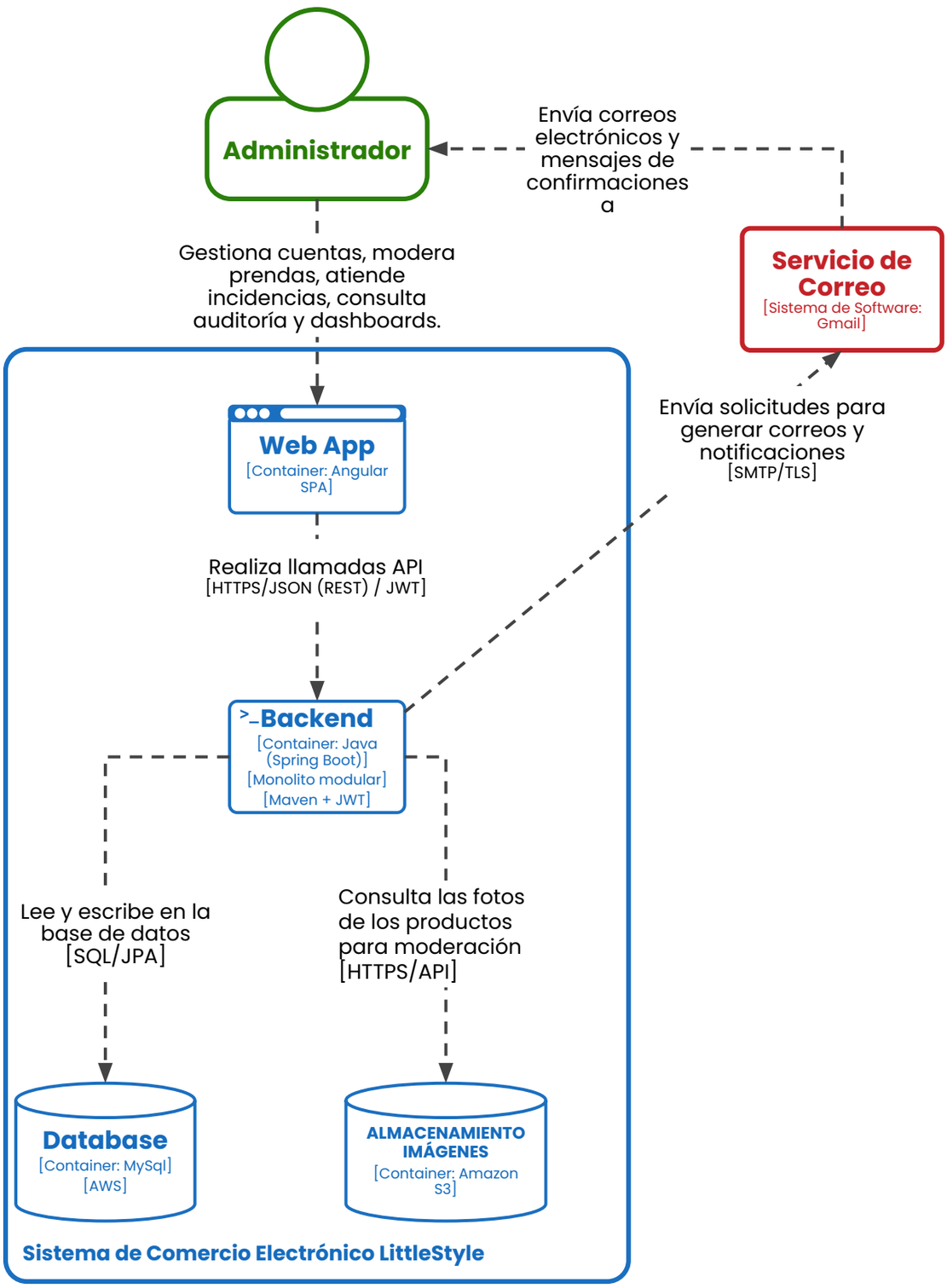
- **Vendedor**



2.6 Diagrama de contenedores (N3)

Presenta "qué partes ejecutables" tiene el sistema LittleStyle (web, API, BD, app móvil, colas, etc.).

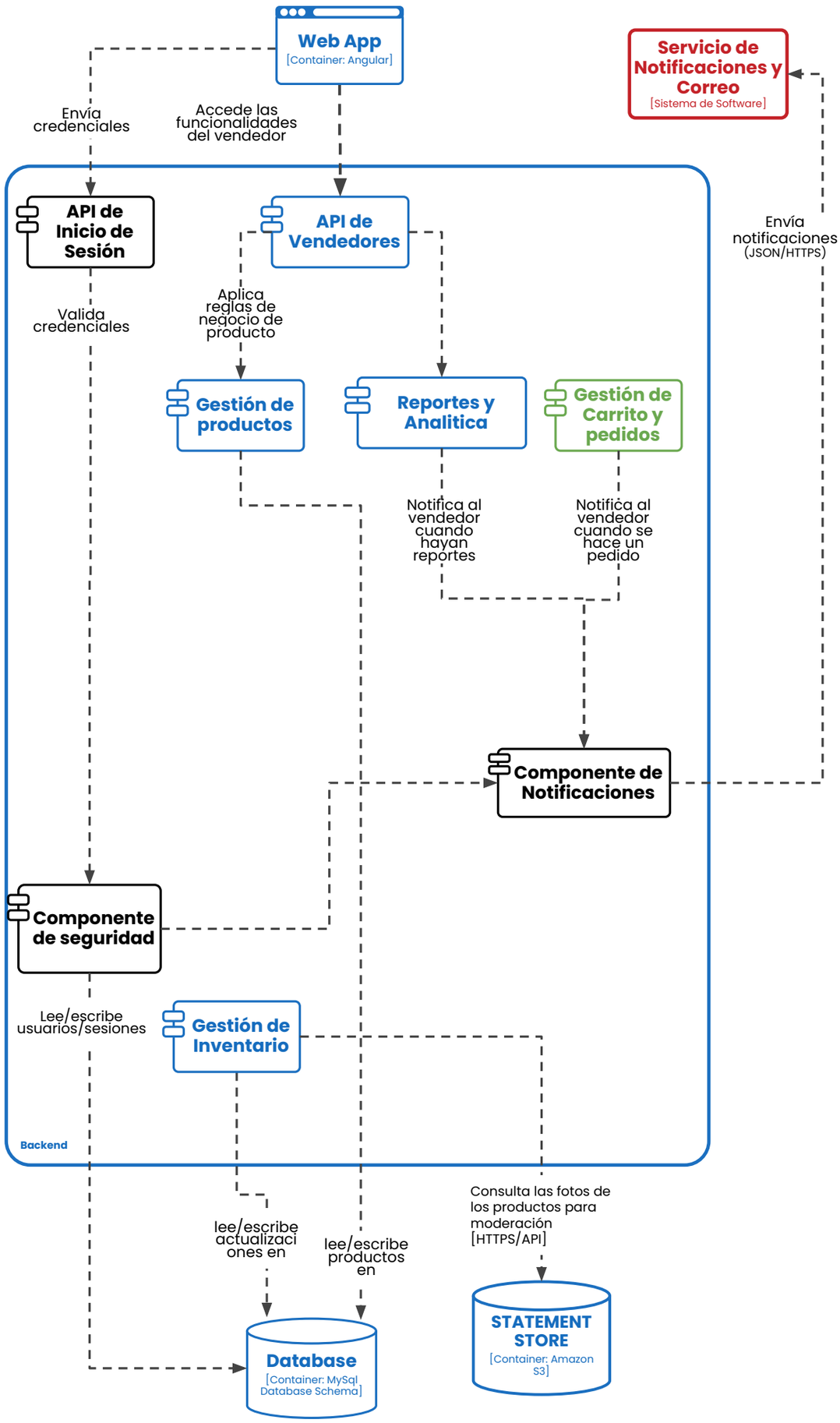
- **Administrador**



2.7 Diagrama de componentes (N1)

Presenta el interior de los componentes de LittleStyle (por ejemplo, el Backend/API). Separados por tipo de usuario

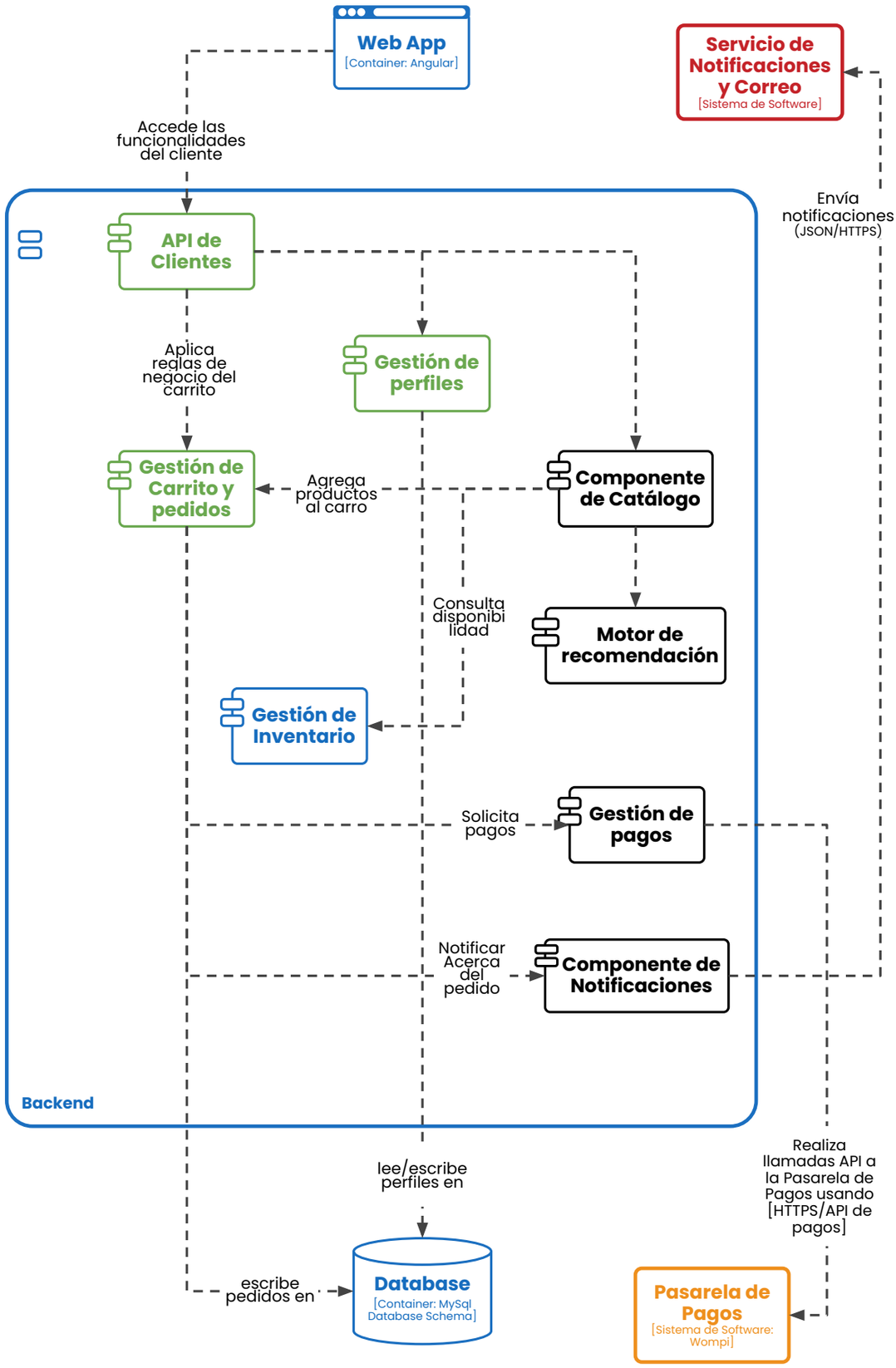
Cliente



2.8 Diagrama de componentes (N2)

Presenta el interior de los componentes de LittleStyle (por ejemplo, el Backend/API). Separados

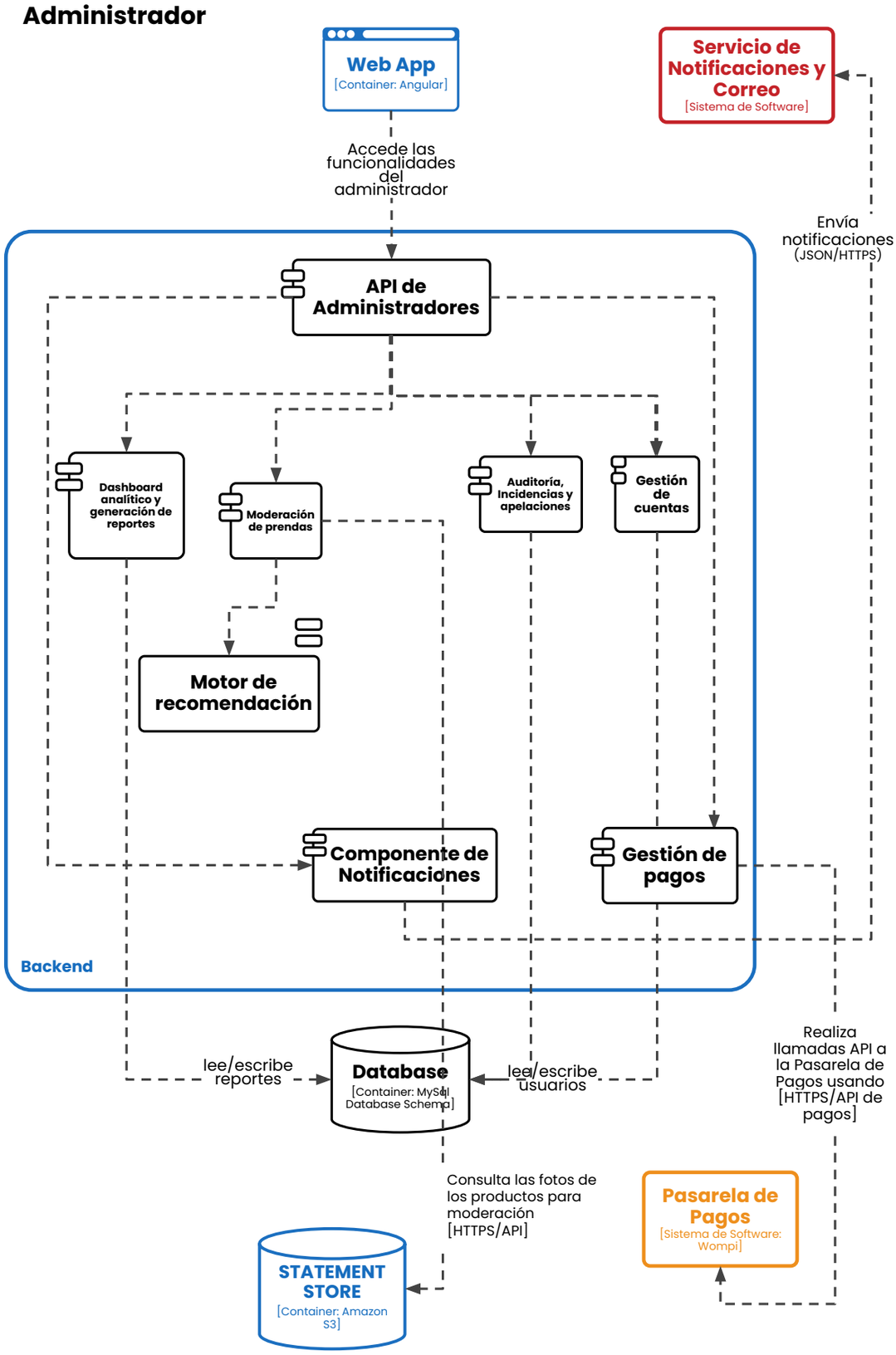
Vendedor



2.9 Diagrama de componentes (N3)

Presenta el interior de los componentes de LittleStyle (por ejemplo, el Backend/API). Separados

Administrador



2.10 Código (N4)

2.10.1 Estructura del repositorio y módulos

El sistema se implementa en dos módulos principales alineados con el Nivel 2 (Contenedores): una SPA web en Angular como canal del cliente y un Backend en Java (Spring Boot) que centraliza la lógica de negocio e integraciones externas.

Estructura (árbol):

```
/frontend/
   /public
   /src
        /app
                /core
                        /guards
                                /auth.guard.ts
                                /role.guard.spec.ts
                                /role.guard.ts
                        /interceptors
                                /auth.interceptor.spec.ts
                                /auth.interceptor.ts
                        /models
                                /auth.models.ts
                        /services
                                /auth.service.spec.ts
                                /auth.service.ts
                        /testing
                                /auth-testing.ts
                /features
                        /admin
                                /models/
                                /pages/
                                /services/
                                admin.routes.ts
                        /auth
                                /components/
                                /pages
                                        /login/
                                        /registro/
                                /auth.routes.ts
                                /auth.validators.ts
                        /catalogoCompras
                                /components/
                                /data/
                                /models/
                                /pages/
                                /services/
                                /testing/
                                /catalogo-compras.routes.ts
                                /catalogo.routes.ts
                        /cuenta/pages/
                        /home/pages
```

2.10 Código (N4)

```
                    /pedidos
                        /models/
                        /pages/
                        /services/
                    /perfilesInfantiles
                        /models
                        /pages/
                        /services/
                    /recomendacion/
                    /vitrina/
            /shared
                    /components/
                    /pages/
                /app.config.ts
                /app.routes.ts
                /app.spec.ts
                /app.ts
        /environments/
        /styles/
        /index.html
        /main.ts
        /styles.scss
        /test-setup.ts
    /.dockerignore
    /.editorconfig
    /.gitignore
    /.prettierrc
    /Caddyfile
    /Dockerfile
    /README.md
    /angular.json
    /package-lock.json
    /package.json
    /proxy.conf.json
    /tsconfig.app.json
    /tsconfig.json
    /tsconfig.spec.json
```

2.10 Código (N4)

Estructura (árbol):

/backend/
- /.mvn/
- /DocumentosArq/
- /deploy/
- /docs/
- /src/main/java/co/edu/uniquindio/littlestyle
  - /config/
  - /modules
    - /admin/controller/
    - /admin
      - /controller/
      - /dto/
      - /model/
      - /repository/
      - /service/
    - /auth
      - /controller/
      - /dto/
      - /model/
      - /repository/
      - /service/
    - /catalogocompras
      - /carrito/
      - /catalogo/
      - /checkout/
      - /inventario/
      - /pago/
      - /recomendacion/
    - /perfilesinfantiles/
      - /controller/
      - /dto/
      - /model/
      - /repository/
      - /service/
    - /perfilesinfantiles/
      - /controller/
      - /dto/
      - /factory/
      - /repository/
      - /service/

2.10 Código (N4)

Estructura (árbol):

/shared/
/LittleStyleApplication.java
/src/main/resources/
/src/test/java/co/edu/uniquindio/littlestyle
/config/
/modules/
/shared/correo/
/LittleStyleApplicationTests.java
/src/test/resources/
/.dockerignore
/.env.example
/.gitattributes
/.gitignore
/Dockerfile
/Jenkinsfile
/README.md
/docker-compose.local.yml
/docker-compose.yml
/mvnw
/mvnw.cmd
/pom.xml

3. DECISIONES DE ARQUITECTURA (ADRs)

ADR-01: Canal de usuario mediante aplicación web frontend en Angular

- **Decisión:** Desarrollar la interfaz cliente/usuario como una Single Page Application (SPA).
- **Contexto:** Los tres usuarios del sistema (Clientes, Vendedores y Administradores) requieren acceder desde exploradores web para gestionar perfiles infantiles, catálogo, inventario, pedidos y dashboards.
- **Alternativas:** Aplicación web tradicional con renderizado en servidor.
- **Justificación:** Permite un marco de trabajo completo con tipado fuerte, inyección de dependencias y manejo de formularios reactivos.
- **Consecuencias:** Se requiere configurar el enrutamiento del lado del cliente, implementar Guards para restringir vistas según el rol y manejar interceptores HTTP para adjuntar el token JWT.

ADR-02: Backend estructurado en una arquitectura Monolítica Modular con Spring Boot

- **Decisión:** Construir el backend central como una aplicación de Monolito Modular utilizando Java con Spring Boot.
- **Contexto:** Se requiere una API REST que gestione la lógica de negocio dividida en módulos bien definidos.
- **Alternativas:** Arquitectura en capas.
- **Justificación:** El monolito modular permite mantener la lógica organizada por paquetes/módulos independientes con límites claros, facilitando el desarrollo y despliegue unificado.
- **Consecuencias:** Se debe mantener una rigurosa separación de dependencias entre módulos dentro del proyecto Java para evitar un alto acoplamiento.

ADR-03: Gestión de dependencias y ciclo de vida del proyecto con Apache Maven

- **Decisión:** Adoptar *Apache Maven* como herramienta de gestión de dependencias y automatización del proceso de compilación (build)
- **Contexto:** El backend requiere integrar múltiples módulos y librerías de Spring (Spring Security, Spring Data JPA, Spring Mail) e integraciones externnas de forma automatizada y estructurada.
- **Alternativas:** Gradle.
- **Justificación:** Maven proporciona una estructura de proyecto estandarizada, una gestión clara mediante el archivo *pom.xml* y una amplia compatibilidad con entornos de integración y despliegue continuo (CI/CD).
- **Consecuencias:** Se debe mantener una correcta gestión de las versiones de dependencias en el archivo pom.xml para prevenir conflictos de transitividad.

3. DECISIONES DE ARQUITECTURA (ADRs)

ADR-04: Autenticación y Autorización basada en JSON Web Tokens (JWT)

- **Decisión:** Implementar un mecanismo de seguridad sin estado (*stateless)* mediante tokens JWT integrados con Spring Security.
- **Contexto:** La API REST del backend debe autenticar solicitudes HTTP y validar el rol del usuario de manera segura y eficiente.
- **Alternativas:** Protocolo OAuth2/OpenID Connect
- **Justificación:** Al ser una API consumida por una SPA en Angular, JWT evita guardar estado de sesión en el servidor, permitiendo validar permisos mediante la firma digital en cada petición sin realizar consultas continuas a la base de datos.
- **Consecuencias:** Los tokens no se pueden revocar antes de su expiración, pero el filtro verifica en cada petición que la cuenta esté activa, por lo que una cuenta suspendida o bloqueada pierde el acceso de inmediato.

ADR-05: Persistencia de datos relacional con MySQL

- **Decisión:** Utilizar MySQL como motor de base de datos relacional.
- **Contexto:** La plataforma debe almacenar información estructurada con relaciones complejas e integridad transaccional.
- **Alternativas:** PostgreSQL.
- **Justificación:** MySQL ofrece un rendimiento óptimo en transacciones ACID.
- **Consecuencias:** Se requiere un diseño eficiente del esquema relacional, con índices adecuados para garantizar tiempos de respuesta rápidos en las en las consultas.

ADR-06: Integración con Pasarela de Pagos Wompi y recepción de eventos mediante Webhook

- **Decisión:** Delegar la recepción y procesamiento de pagos con tarjetas/PSE a la pasarela Wompi, sincronizando el estado de las transacciones mediante un endpoint Webhook en el backend.
- **Contexto:** Los usuarios deben pagar sus pedidos dentro del flujo de compra y la plataforma debe actualizar automáticamente el estado del pedido.
- **Alternativas:** Verificación manual o por consulta periódica. Procesamiento directo de datos de tarjeta en el backend (se requiere certificación PCI-DSS)
- **Justificación:** Wompi abstra el cumplimiento de PCI-DSS para el manejo de tarjetas. La arquitectura orientada a Webhooks permite que Wompi notifique en tiempo real al backend cuando un pago es aprobado, garantizando la actualización automática de pedidos e inventario.
- **Consecuencias:** El backend debe validar la firma de seguridad del Webhook para evitar peticiones maliciosas y controlar la re-notificación de eventos duplicados.

3. DECISIONES DE ARQUITECTURA (ADRs)

ADR-07: Notificaciones transaccionales vía correo electrónico con Servicio Gmail (SMTP)

- **Decisión:** Integrar el módulo de notificaciones transaccionales del backend con el Servicio de correo de Gmail mediante protocolo SMTP.
- **Contexto:** El sistema debe notificar por correo electrónico a los Clientes sobre la confirmación de compras y el envío del paquete con su número de guía, asi como emitir alertas de stock bajo a los Vendedores.
- **Alternativas:** Amazon SES, SendGrid, Mailgun.
- **Justificación:** Facilita la configuración y pruebas de desarrollo sin incurrir en costos iniciales de servicios transaccionales dedicados, permitiendo enviar plantillas de correo automatizadas desde Spring Boot.
- **Consecuencias:** Se deben gestionar contraseñas de aplicación o credenciales de forma segura (variables de entorno) y tener en cuenta los límites diarios de envío del servicio de Gmail.

ADR-08: Acceso a datos mediante Mapeo Objeto-Relacional con Spring Data JPA / Hibernate

- **Decisión:** Utilizar Spring DATA JPA (con Hibernate como proveedor ORM) para la capa de accesos a datos del backend.
- **Contexto:** El backend necesita interactuar con MySQL para realizar operaciones CRUD y consultas complejas de manera segura y abstracta.
- **Alternativas:** JDBC Template puro, SQL nativo en repositorios. MyBatis.
- **Justificación:** Reduce el código repetitivo, automatiza el mapeo de tablas a entidades Java, previene vulnerabilidad de Inyección SQL y proporciona soporte nativo para paginación y ordenamiento.
- **Consecuencias:** Se debe supervisar el rendimiento de las consultas y configurar adecuadamente la carga perezosa para evitar problemas de rendimiento.

3. DECISIONES DE ARQUITECTURA (ADRs)

ADR-09: Amazon Web Services (AWS) como Plataforma de Despliegue

- **Decisión:** Seleccionar Amazon Web Services (AWS) como el proveedor principal de infraestructura en la nube para el alojamiento, despliegue y gestión de recursos del sistema LittleStyle.
- **Contexto:** La plataforma requiere una infraestructura en la nube altamente disponible, segura y escalable para alojar la base de datos y ejecutar las instancias del backend y frontend.
- **Alternativas:** Microsoft Azure, Google Cloud Platform.
- **Justificación:** AWS ofrece integración nativa de diferentes servicios simplificando la configuración de despliegue automatizado desde jenkins. Además cuenta con alta disponibilidad global y herramientas de gestión de identidad muy robustas.
- **Consecuencias:** Se debe configurar AWS IAM para gestionar roles y permisos de forma segura, definir variables de entorno protegidas dentro de los pipelines de CI/CD para evitar la exposición de credenciales.

ADR-10: Almacenamiento multimedia externo en Amazon S3 y referencia mediante URLs en Base de Datos.

- **Decisión:** Seleccionar Amazon S3 como la plataforma de almacenamiento de objetos externos para alojar las imágenes del catálogo, guardando exclusivamente las direcciones URL en la base de datos.
- **Contexto:** La plataforma requiere almacenar y servir archivos multimedia de forma rápida sin sobrecargar el motor relacional de base de datos ni saturar el servidor de aplicaciones backend con el envío de archivos estáticos.
- **Alternativas:** Cloudflare R2, Backblaze B2.
- **Justificación:** Amazon S3 se integra de forma nativa con la infraestructura principal alojada en AWS. Delega la descarga directa de imágenes desde S3 hacia el cliente en Angular optimizando el uso del ancho de banda del backend en Spring Boot.
- **Consecuencias:** El backend en Spring Boot debe incluir el SDK de AWS para procesar la subida, validación y borrado de imágenes. Se deben configurar políticas de lectura pública en el bucket S3 para servir los archivos directamente sobre HTTPS.

3. DECISIONES DE ARQUITECTURA (ADRs)

ADR-11: Selección de JasperReports para PDF y Apache POI para XLSX EN LA Exportación de Reportes.

- **Decisión:** Integrar JasperReports 7.0.6 en el backend para generar los reportes PDF a partir de plantillas JRXML (ventas, demanda, inventario y búsquedas). Usar Apache POI 5.4.1 para generar archivos XLSX con las hojas Datos y Resumen. Generar el CSV sin librería externa, en UTF-8 con BOM, separador ; y saltos de línea CRLF.
- **Contexto:** Vendedores y administradores necesitan exportar consolidados de ventas, demanda por talla, inventario y búsquedas del catálogo para analizarlos fuera de la plataforma. Cada formato responde a un uso distinto: PDF para presentación, XLSX para análisis con tipos numéricos y CSV para intercambio con otras herramientas.
- **Alternativas:** Text y Apache PDFBox para el PDF. Maquetar los archivos directamente desde el código.
- **Justificación:** JasperReports separa el diseño de la plantilla del código fuente, sin coordenadas ni píxeles, y se alimenta de colecciones de DTOs. Apache POI es la librería estándar para generar XLSX con celdas tipadas. El CSV no requiere dependencia adicional.
- **Consecuencias:** Se agregan dos dependencias al pom.xml. Las plantillas se compilan en el arranque de la aplicación. Cada reporte tiene un límite de 5.000 filas, validado antes de cargar los datos en memoria (respuesta 422 si se excede). Caddy no comprime /api/v1/reportes/* para conservar el encabezado Content-Length de la barra de progreso.

ADR-12: Patrón de Diseño Strategy para el Motor de Recomendaciones de Tallas y Filtrado de Sensibilidad Textil

- **Decisión:** Implementar el patrón de diseño Strategy en la capa de servicio del backend para desacoplar las reglas de cálculo biométrico de tallas y los algoritmos de filtrado/exclusión de telas por alergias sensibles.
- **Contexto:** El proceso de negocio uno requiere evaluar las medidas de los niños cruzadas con las tablas del fabricante, así como aplicar restricciones por alergias textiles. Estas reglas pueden variar según la categoría del producto o la marca.
- **Alternativas:** Condiciones complejas como if-else o switch centralizadas en una sola clase de servicio.
- **Justificación:** Permite encapsular cada algoritmo de recomendación y regla de filtrado en clases independientes que implementan una interfaz común. Esto facilita la adición o modificación de algoritmos sin alterar el código existente.
- **Consecuencias:** Aumenta ligeramente la cantidad de clases pero elimina el acoplamiento y previene errores de regresión.

3. DECISIONES DE ARQUITECTURA (ADRs)

ADR-13: Patrón Data Transfer Object (DTO) para el Desacoplamiento entre Entidades de Dominio y la API REST

- **Decisión:** Adoptar el patrón DTO en la totalidad de las comunicaciones entre el frontend y los controladores REST del backend.
- **Contexto:** La base de datos gestiona entidades complejas con relaciones ORM. exponer estas entidades directamente a la API REST genera riesgos de seguridad y cuellos de botella por serialización.
- **Alternativas:** Serialización y exposición directa de las entidades JPA anotadas con @Entity hacia el frontend.
- **Justificación:** Oculta campos sensibles de la base de datos garantizando el cumplimineto de Habeas Data. Mantiene la compatibilidad estandarizada de contratos JSON entre Angular y Spring Boot.
- **Consecuencias:** Se requiere implementar mapeadores para convertir entre DTOs y Entidades de JPA.

ADR-14: Patrón Observer / Event-Driven para pagos, notificaciones y avisos administrativos

- **Decisión:** Usar el patrón Observer con eventos de Spring (@TransactionalEventListener, fase AFTER_COMMIT) para ejecutar de forma asíncrona y desacoplada las acciones derivadas de un hecho de negocio ya confirmado.
- **Contexto:** Cuando Wompi confirma un pago mediante un Webhook, el sistema debe actualizar el pedido, descontar el stock y notificar por correo. De forma análoga, los cambios de estado de cuenta, la moderación de prendas y el cierre de incidencias deben avisar por correo a la persona afectada (NotificacionAdministracionListener).
- **Alternativas:** Ejecución secuencial dentro del controlador o del servicio que origina el cambio.
- **Justificación:** Desacopla la acción principal de las tareas secundarias. Si el servicio de correo presenta lentitud o falla, no afecta la confirmación del pago, el cambio de estado de la cuenta ni la actualización del inventario. Al ejecutarse después del commit, nunca se envía un aviso de un cambio que luego se revierta.
- **Consecuencias:** Los listeners corren fuera de la transacción original, por lo que un fallo de correo se registra pero no revierte la operación. Se requieren políticas de reintento en el envío de correo.

3. DECISIONES DE ARQUITECTURA (ADRs)

ADR-15: Patrón Facade para la Coordinación del Proceso de Checkout y Creación de Pedidos

- **Decisión:** Implementar un servicio facade en el módulo de catálogo y compras que organice la interacción entre los submódulos de perfiles, inventario, catálogo, pasarela de pago y pedidos.
- **Contexto:** El flujo de compra involucra validar la disponibilidad de productos en inventario, consultar restricciones del perfil infantil activo, liquidar subtotales, costo de envío y comunicarse con la pasarela de pagos.
- **Alternativas:** Permitir que los controladores HTTP consuman e interconectan directamente múltiples repositorios y servicios de diferentes módulos.
- **Justificación:** Ofrece una interfaz simplificada y unificada para el controlador de checkout. Oculta la complejidad de la interacción entre subsistemas y mantiene la separación estricta de responsabilidades en la arquitectura monolítica modular.
- **Consecuencias:** La clase Facade debe mantenerse cohesionada para no volverse una clase con exceso de responsabilidad (God Object).

ADR-16: Patrón Factory Method para el Módulo de Generación y Exportación de Reportes Analíticos

- **Decisión:** Aplicar el patrón Factory Method en el módulo reportes para crear el generador de cada formato. Cada valor de FormatoReporte (PDF, XLSX, CSV) se asocia a su creador (CreadorGeneradorPdf, CreadorGeneradorXlsx, CreadorGeneradorCsv), que produce su generador. GeneradorReporteFactory resuelve el creador según el parámetro formato de la petición.
- **Contexto:** Administradores y vendedores exportan los mismos reportes en varios formatos estructurados desde GET /api/v1/reportes/{tipo}.
- **Alternativas:** Estructuras condicionales (if/switch) dentro del controlador o del servicio para formatear la respuesta en cada formato.
- **Justificación:** Centraliza la creación del exportador y mantiene el controlador sin lógica por formato. Cumple los principios abierto/cerrado y de responsabilidad única: agregar un formato nuevo no modifica los controladores.
- **Consecuencias:** Se requiere una interfaz común (GeneradorReporte) que estandarice la construcción del archivo descargable. Se agregan clases por cada formato.

3. DECISIONES DE ARQUITECTURA (ADRs)

ADR-17: Patrón Interceptor / Chain of Responsability para la Seguridad y Manejo de Peticiones

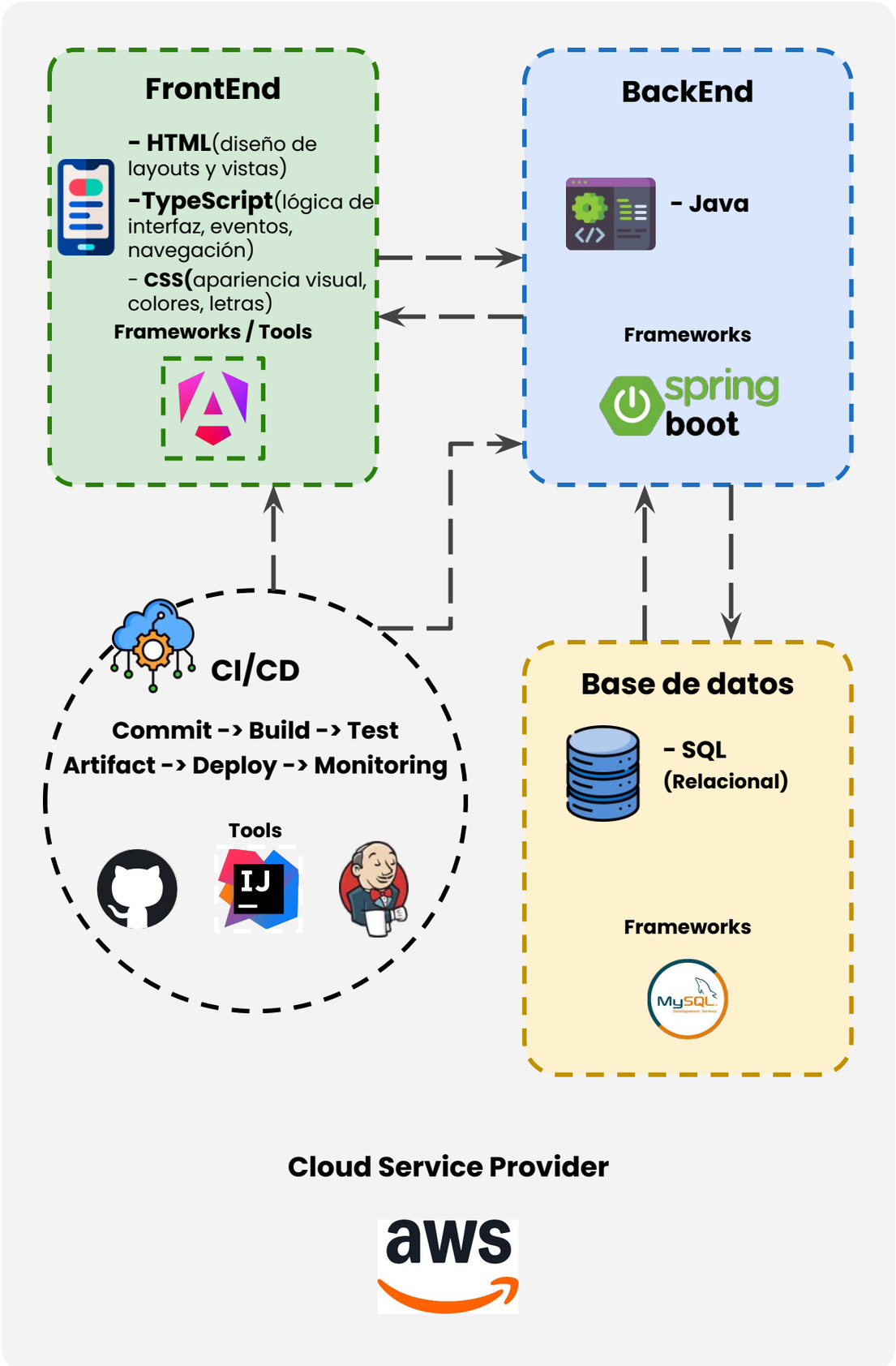
- **Decisión:** Implementar el patrón Chain of Responsability mediante la cadena de filtros de Spring Security en el backend y HTTP Interceptors en Angular. El filtro JWT, además de validar la firma y el rol, consulta el estado de la cuenta (EstadoCuenta) y rechaza la petición si está suspendida, bloqueada o eliminada.
- **Contexto:** Cada petición recibida por la API debe ser interceptada para validar el token JWT, verificar los permisos del rol y sanitizar las entradas contra inyecciones SQL. En el frontend, cada solicitud saliente debe adjuntar automáticamente el encabezado.
- **Alternativas:** Validar manualmente la autenticación y permisos dentro de cada método de cada controlador REST.
- **Justificación:** Permite procesar de forma transversal e independiente los aspectos de seguridad antes de que la petición llegue a la lógica de negocio.
- **Consecuencias:** Requiere una configuración rigurosa en Spring Security para evitar bloqueos no deseados en endpoints públicos.

ADR-18: Auditoria administrativa append-only con cadena de hashes SHA-256**

- **Decisión:** Registrar cada acción administrativa en la tabla de auditoria como un evento inmutable y encadenar los registros con SHA-256
- **Contexto:** Las acciones sobre cuentas, publicaciones, incidencias y pedidos requieren no repudio y detección de manipulación posterior
- **Alternativas:** Tabla de log simple sin encadenamiento o auditoría delegada a un servicio externo.
- **Justificación:** El encadenamiento permite detectar cualquier alteración o borrado de un registro intermedio, sin depender de un servicio externo
- **Consecuencias:** Toda acción administrativa debe registrarse en la misma operación de negocio, los intentos fallidos se registran como FALLO

4. STACK TECNOLÓGICO DEL SISTEMA

En esta sección se describen las tecnologías utilizadas para la implementación del sistema LittleStyle, organizadas por capa: cliente web (frontend), backend y base de datos. Esta descripción es coherente con la arquitectura presentada en los diagramas C4 y corresponde al alcance definido para el rol del cliente.



VIGILADA MINEDUCACIÓN



UNIQUINDÍO, en conexión territorial

Carrera 15 Calle 12 Norte Tel: (606) 7 35 93 00 Armenia - Quindío - Colombia

www.uniquindio.edu.co